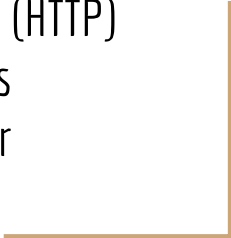


Web & apps

Hypertext Transfer Protocol (HTTP)

Konstantinos Tserpes

tserpes@mail.ntua.gr



Objectives

- Define Web
- Remember the HTTP basics
- Establish a common terminology

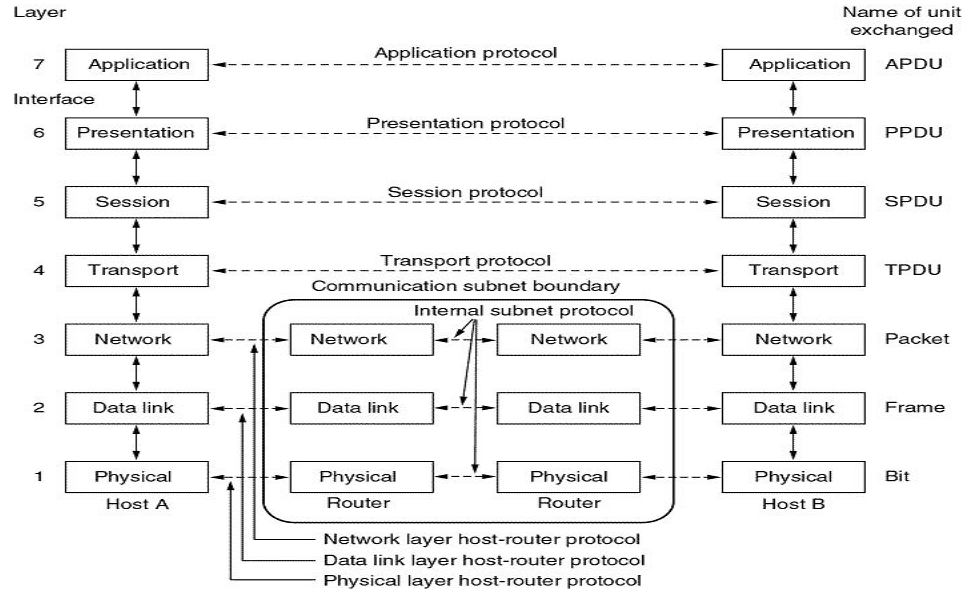


Internet

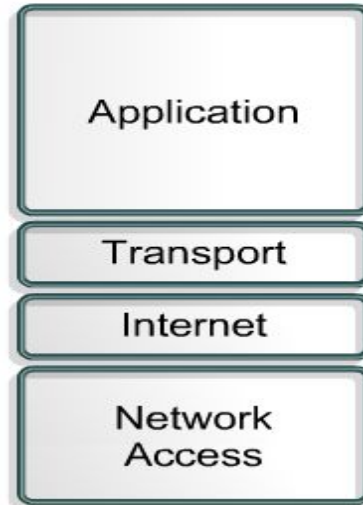
RECALL...

- ISO/OSI
- TCP/IP

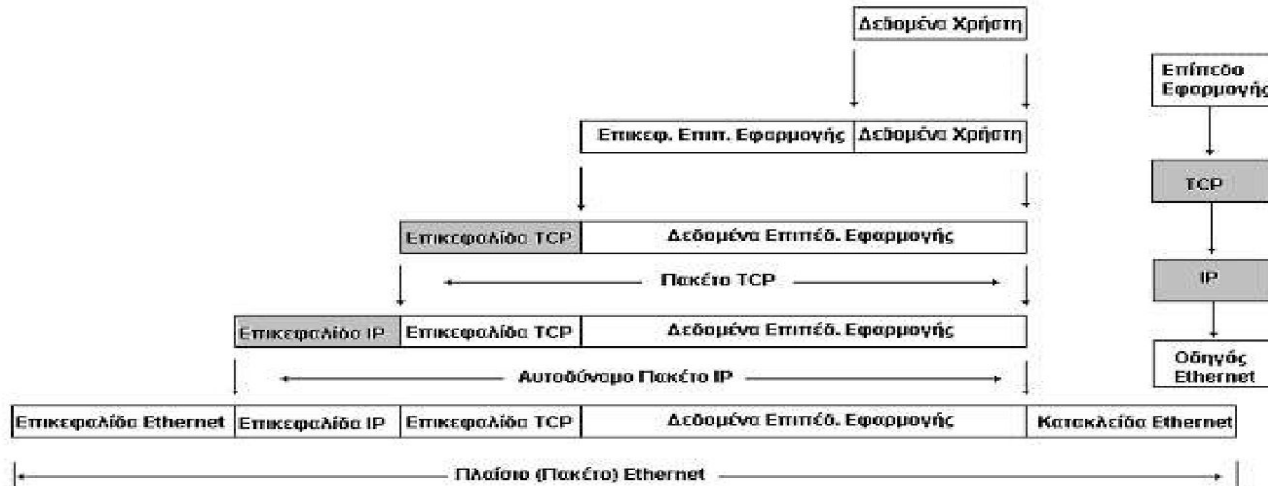
ISO/OSI



TCP/IP

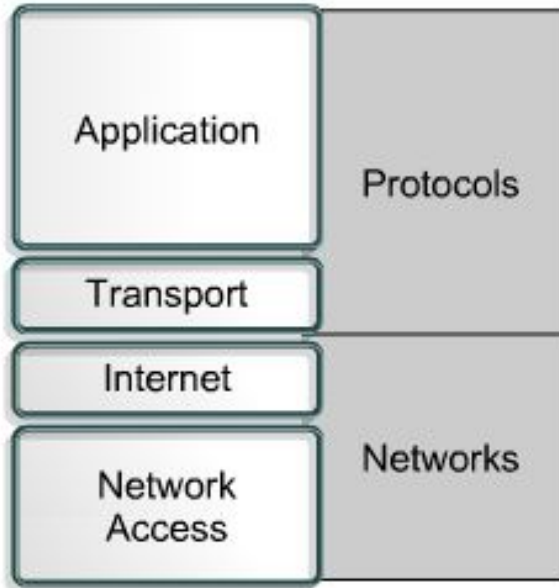


Transport and packets

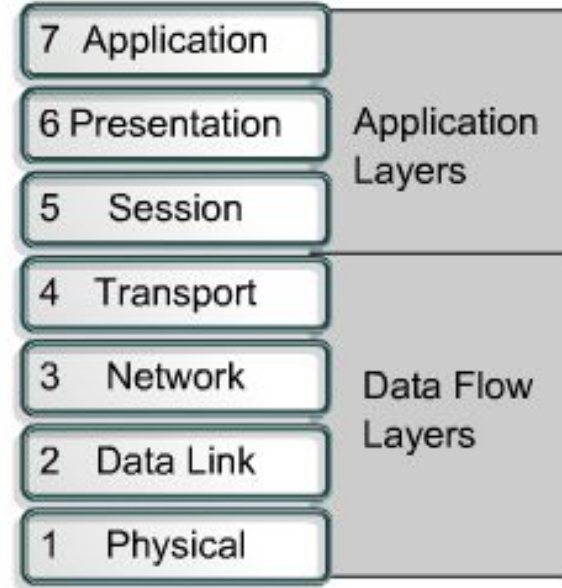


TCP/IP Vs ISO/OSI

TCP/IP Model



OSI Model



www

Context

- Story:
 - CERN 1989
 - Internet existed
 - FTP was a very popular protocol
 - Scientists looking for a way to exchange documents
- System to efficiently exchange documents
 - Accessibility: Work across various platforms (UNIX, VMS, Macintosh, Windows) without relying on proprietary software
 - Heterogeneity: Handle different data formats and link them seamlessly
 - Non-linear information access: follow related topics via links rather than reading documents sequentially
 - Decentralization: No central approval or coordination was required for adding new information
 - Ease of use: user-friendly and easy to adopt without extensive training
 - Scalability: Accommodate new documents and users without major overhauls
 - Robustness and resilience: Reliable, always-on access to critical documentation
 - No central editing authority: Documents updated and linked by various users without requiring approval from a central gatekeeper.

Implementation

- Tim Berners-Lee addressed these requirements by developing:
 - A standard document format for hypertext
 - A standardised protocol that facilitates efficient retrieval of documents stored on a server
 - A standardised addressing scheme that allows unambiguous identification of documents
- Originally called “*mesh*” but then adopted “*world wide web*”
- The World Wide Web was simple, open, and extensible

Implementation

- Tim Berners-Lee addressed these requirements by developing:
 - A standard document format for hypertext
 - A standardised protocol that facilitates efficient retrieval of documents stored on a server
 - A standardised addressing scheme that allows unambiguous identification of documents
- Originally called “*mesh*” but then adopted “*world wide web*”
- The World Wide Web was simple, open, and extensible



Our focus today

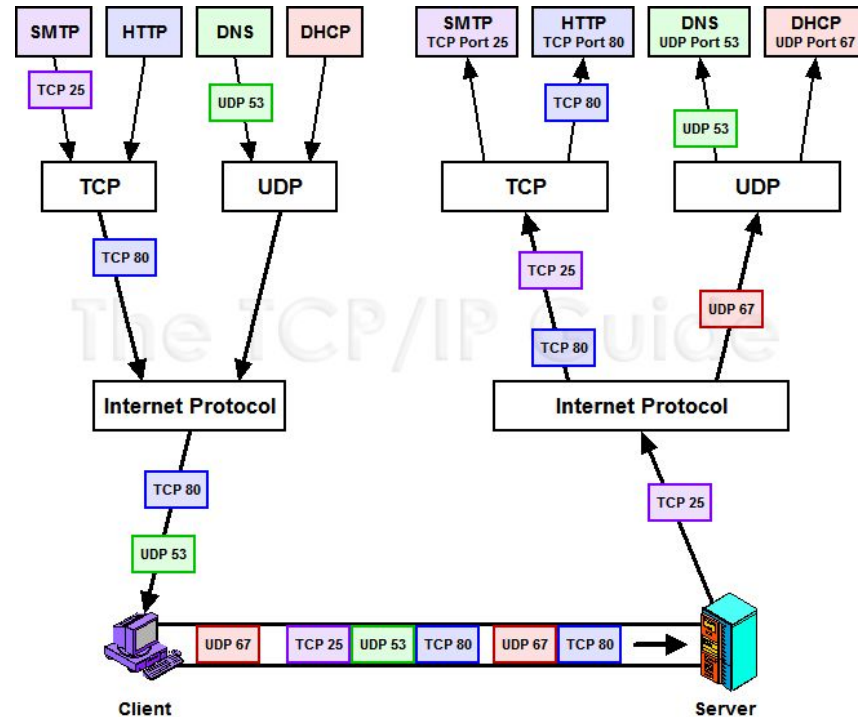
A diagram consisting of two thin grey arrows originates from a brown rounded rectangle labeled 'Our focus today'. One arrow points to the third bullet point, 'The World Wide Web was simple, open, and extensible'. The other arrow points to the third sub-bullet point, 'A standardised addressing scheme that allows unambiguous identification of documents'.

HTTP

Hypertext is text displayed on a computer display or other electronic devices with references (hyperlinks) to other text which the reader can immediately access. - *en.wikipedia.org/wiki/Hypertext*

- HyperText Transfer Protocol
- Application text-based level protocol for the exchange of hypertext document
- Standardizes
 - Resource names (URL)
 - Requests
 - Responses
- Versions: HTTP/0.9, 1.0, 1.1, 2.0, 3.0
- Ref: Tim Berners Lee, Request for Comment 1945, HTTP/1.0
 - <http://www.w3.org/Protocols/rfc1945/rfc1945>

HTTP Interfacing with the Internet



HTTP in a client-server architecture

- **Client**

- An application program that establishes connections for the purpose of sending **requests**.

- **Server**

- An application program that accepts connections in order to service requests by sending back **responses**

- **User agent**

- The client which initiates a request. These are often **browsers**, editors, spiders (web-traversing robots), or other end user tools

- **Origin server**

- The server on which a given resource resides or is to be created

- **(Web) Resource**

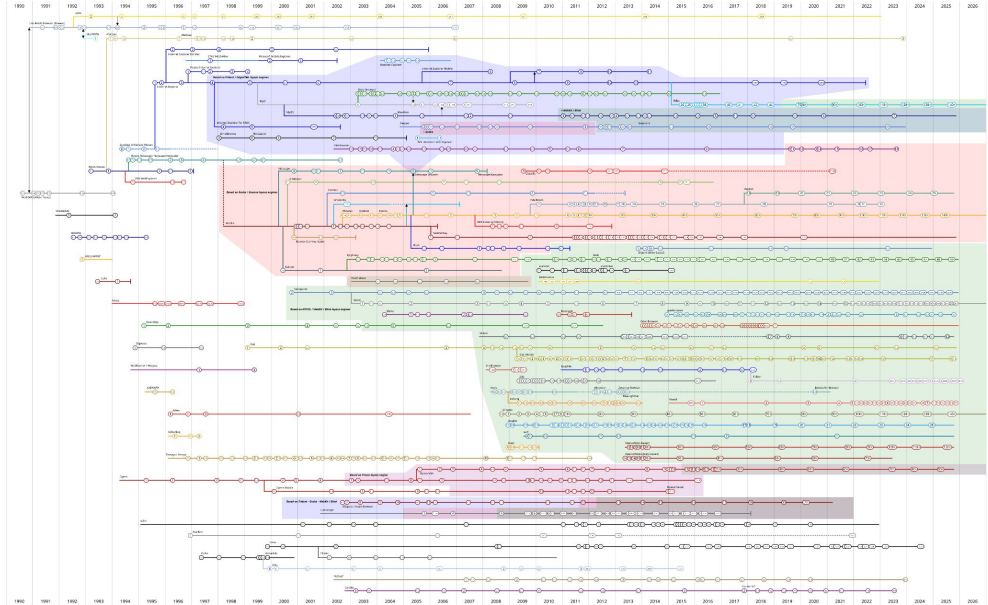
- Any identifiable thing, whether digital, physical, or abstract.

(Web) Resources

- Early definition: static addressable documents or files (August 1998, RFC 2396)
- Current definition: *"every 'thing' or entity that can be identified, named, addressed or handled, in any way whatsoever, in the web at large, or in any networked information system."* (January 2005, RFC 3986)
- This last definition accommodates concepts from Semantic Web
- Examples:
 - Files, Devices, Values, Operands, scripts, web services, databases extracts, ...

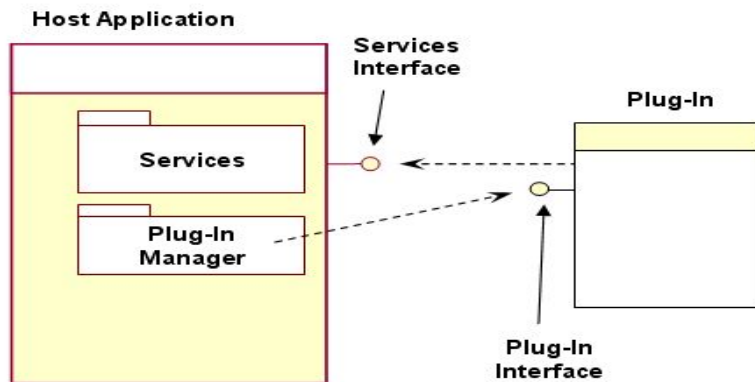
The HTTP browser

- Sends HTTP requests to a server
- Receives and interprets responses
- Visualizes resources
- Timeline



Browser features

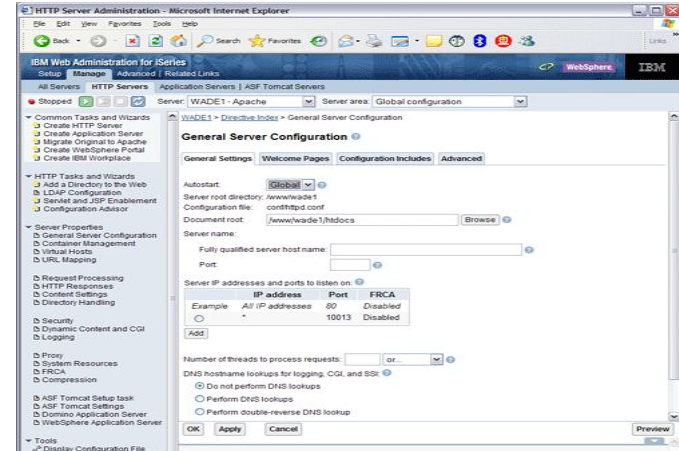
- Version of the document description languages supported (HTML, CSS)
- Native programming language support (Javascript)
- Extension mechanisms
 - Plug-in interface
 - Content viewers (e.g., Adobe Acrobat for PDF, Microsoft Silverlight, Apple Quicktime)
 - Programming language interpreters (e.g., Java)



The HTTP server

- Functionality

- Network access with HTTP for handling requests
- Access to resources in secondary storage
- Delivery of HTTP responses
- Access control
- Server-side program execution
- Logging
- Monitoring and administration
- Virtual hosting
- URL mapping
- Connection to application servers



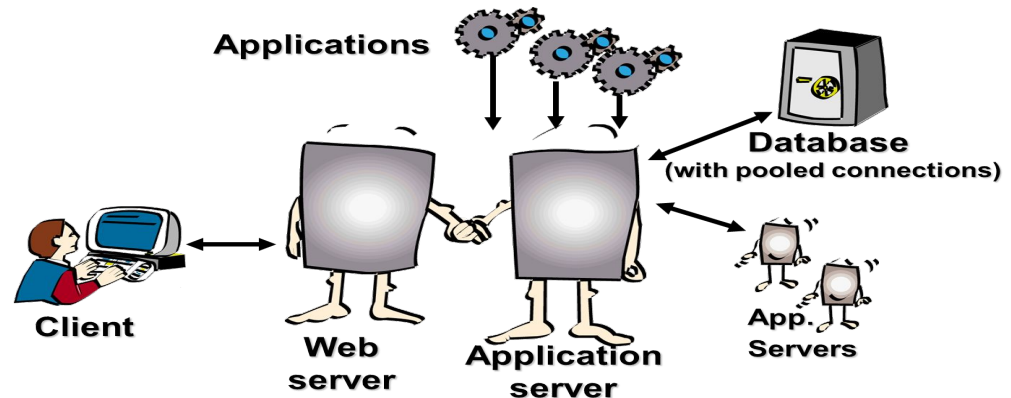
HTTP server Vs application server

HTTP Server

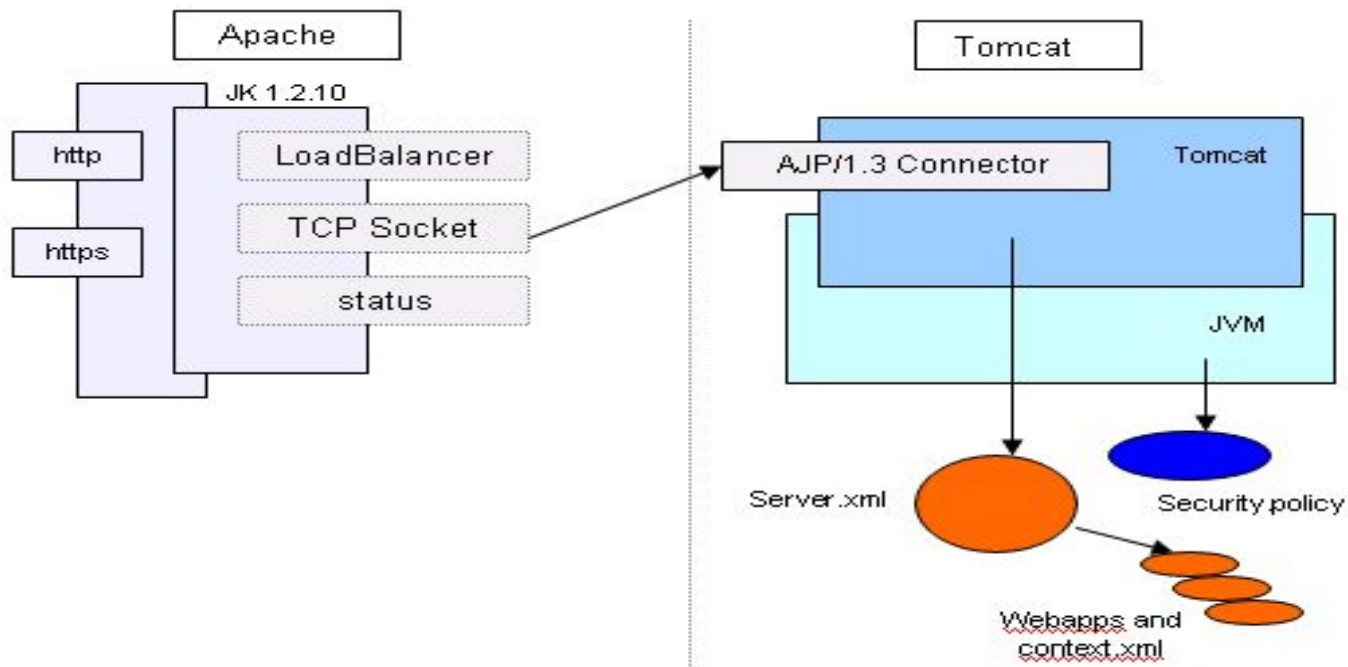
Primary Function: handle HTTP requests and serve static web content (e.g., HTML, CSS, JavaScript, images, and other files) to clients, such as web browsers.

Application Server

Primary Function: execute application logic, process dynamic content, and manage the business logic of web applications.



Example



HTTP characteristics

- HTTP is **stateless**
 - Every HTTP request-response cycle is independent
 - No data are preserved between two connections of the same client or of different clients
 - HTTP is thus **sessionless**
 - HTTP 1.0 also **closes the TCP connection** between the client and the server host at each roundtrip (fixed in HTTP 1.1)

Application server features

- The application server can be **stateful** (e.g. a residential process)
- It can preserve the **user's session** across multiple request-response cycles
- Can preserve session data
- Can handle shared resources (e.g, pool of database connections)
- Can be optimized (multi-threading, multi-processing, multi-host distribution)
- Can be multi-protocol (e.g., Corba IIOP, COM/DCOM)



HTTP Structure



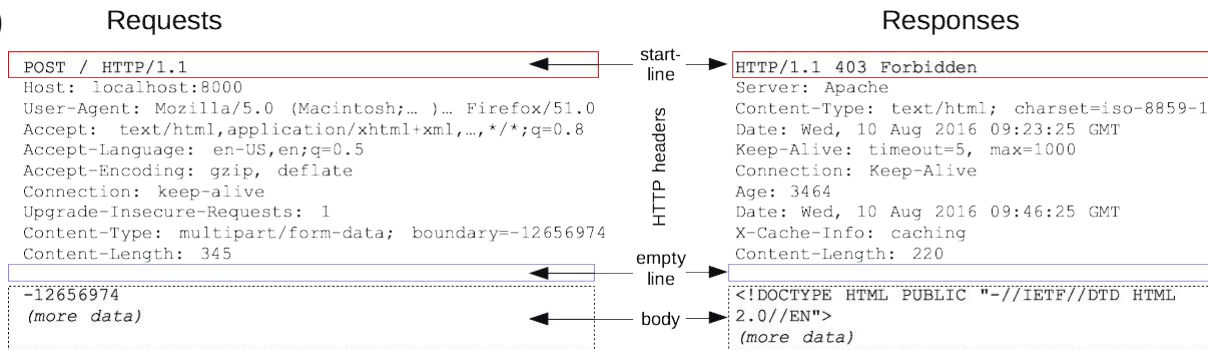
HTTP Conversation

- Client Request

- Request Line: {METHOD} {Resource ID} {HTTP Version}
- Client Generated Headers
- Request Body

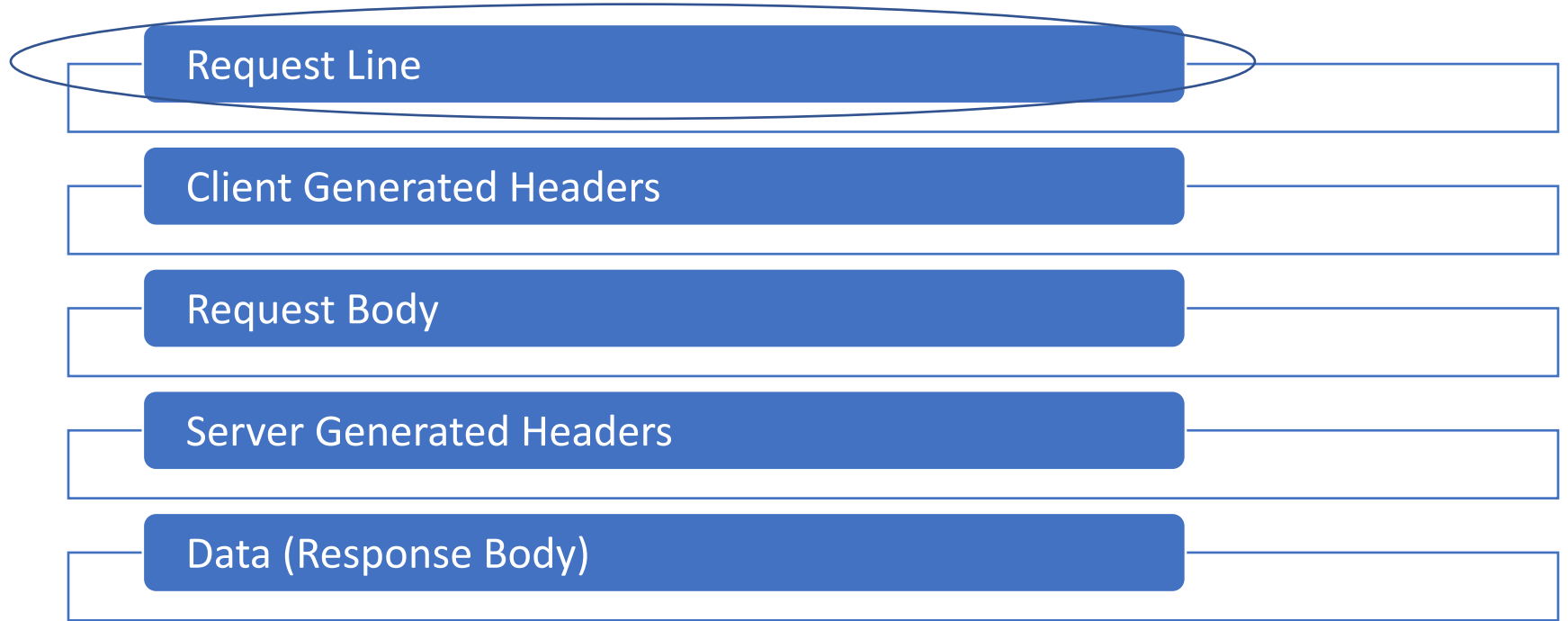
- Server Response

- Status Line
- Server Generated Headers
- Data (Response Body)



Each HTTP header is encoded as a single ASCII-line terminated by CR and LF

Request Line



Request Line

- Syntax: {method}{request target}{protocol}

Methods

- Client commands to the server
- Defined in the HTTP specification
- The implementation of each method depends on the server
- Servers are not expected to implement all methods

Method list

- GET
 - Request for the receipt of a resource
- POST
 - Sending data to a resource
- HEAD
 - Request for the receipt of the response header
- PUT
 - Request for the creation of a resource
- DELETE
 - Request for the deletion of a resource
- CONNECT
 - Establishes a tunnel to the server identified by the target resource (useful when the server is behind a proxy and they need to establish a pure data exchange session)
- TRACE
 - Request to the server to send some data meant for debugging
- OPTIONS
 - Request for the list of supported methods
- PATCH
 - The PATCH method applies partial modifications to a resource.

Protocol

- HTTP 1.0 (1996)
- HTTP 1.1 (1997)
- HTTP/2.0 (2015)
- HTTP/3.0 (2022)

Feature	HTTP/1.1 (1997)	HTTP/2 (2015)	HTTP/3 (2022)
Transport	TCP	TCP	QUIC (UDP)
Performance	Slow (Head-of-line blocking)	Fast (Multiplexing)	Fastest (Reduced latency)
Connections	Persistent connections	Multiplexed (Multiple requests/stream)	Multiplexed (No TCP HOL blocking)
Headers	Plain text	Binary compressed (HPACK)	Binary compressed (QPACK)
Usage	Common, legacy	High-traffic sites	Modern mobile/web

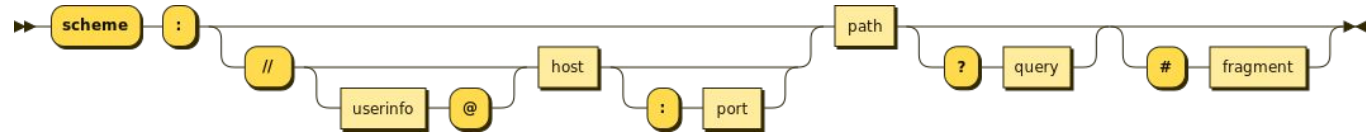
Resource target

- Challenge
 - Identify resources across the whole internet
- Two parts
 - Identification of resource within server
 - Identification of server in the network

Uniform Resource Identifier (URI)

- String of characters that unambiguously identifies a particular resource
- Syntax defined in RFC 3986, January 2005
- Two forms
 - Uniform Resource Name (URN): URI that identifies a resource by name in a particular namespace. E.g. `urn:isbn:0-486-27557-4`
 - Uniform Resource Locator (URL): URI that specifies both its primary access mechanism and network location. E.g. `http://www.ece.ntua.gr/`
- ASCII encoded
 - Other characters and delimiters are percent-encoded (e.g. '#' is %23 or '/' is %2F or 'ω' is %CF%89)

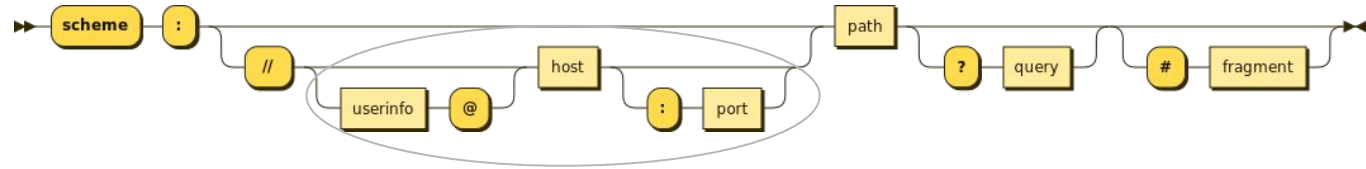
Syntax details



By OmenBreeze - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=82827943>

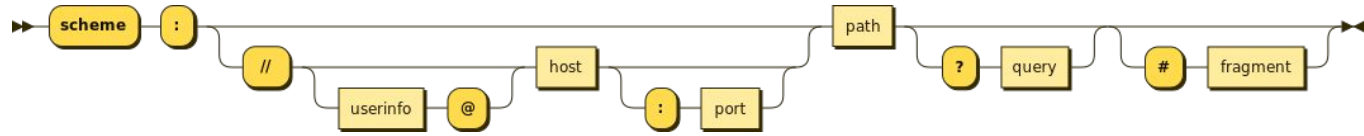
- Scheme
 - Define the type of resource
 - Examples: http, https, ftp, mailto, file, data, irc

Syntax details



- Authority (optional):
 - Userinfo: user name and optional password preceded by a colon (:), followed by an at symbol (@)
 - Host: either a registered name or an IP address
 - port: number between 1 and 65535 inclusive that signifies a logical gate in a device preceded by a colon (:).

Syntax details



- Path: sequence of path segments separated by a slash (/).
 - Query: query string of non-hierarchical data (attribute–value pairs by convention separated by a delimiter, often called: *query string parameters*)
 - Fragment: fragment identifier providing direction to a secondary resource, such as a section heading in an article

Example URIs

The diagram illustrates the hierarchical structure of various URIs. Each URI is shown with brackets indicating its components and their labels:

- https://john.doe@www.example.com:123/forum/questions/?tag=networking&order=newest#top**
 - scheme: https
 - authority: john.doe@www.example.com (userinfo: john.doe, host: www.example.com, port: 123)
 - path: /forum/questions/
 - query: ?tag=networking&order=newest
 - fragment: #top
- ldap://[2001:db8::7]/c=GB?objectClass=one**
 - scheme: ldap
 - authority: [2001:db8::7]
 - path: /c=GB
 - query: ?objectClass=one
- mailto:John.Doe@example.com**
 - scheme: mailto
 - path: John.Doe@example.com
- news:comp.infosystems.www.servers.unix**
 - scheme: news
 - path: comp.infosystems.www.servers.unix
- tel:+1-816-555-1212**
 - scheme: tel
 - path: +1-816-555-1212
- telnet://192.0.2.16:80/**
 - scheme: telnet
 - authority: 192.0.2.16
 - path: /:80/
- urn:oasis:names:specification:docbook:dtd:xml:4.1.2**
 - scheme: urn
 - path: oasis:names:specification:docbook:dtd:xml:4.1.2

Client Generated Headers



Request Line

Client Generated Headers

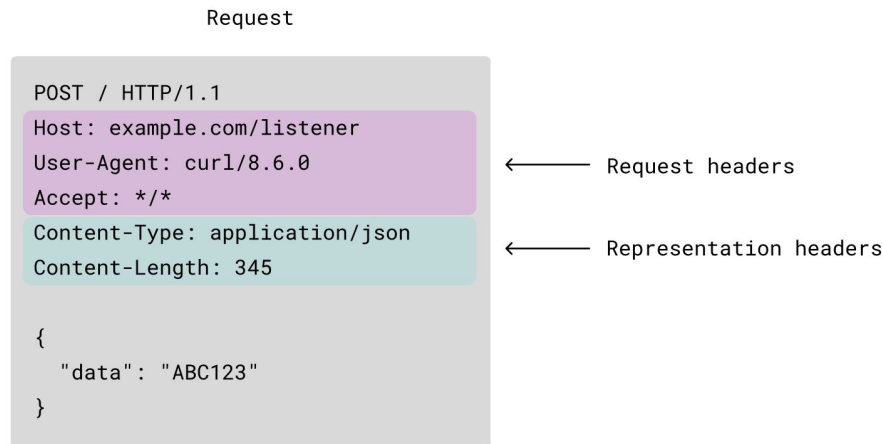
Request Body

Server Generated Headers

Data (Response Body)

Client generated headers

- Request headers provide additional context to a request or add extra logic to how it should be treated by a server
- Representation headers are sent in a request if the message has a body, and they describe the original form of the message data and any encoding applied



Multipurpose Internet Mail Extensions (MIME)

Types - Data Types

- Context
 - Exchange non-ASCII-based documents (Text or Binary)
 - The IETF developed the Multipurpose Internet Mail Extensions (MIME)
- A MIME type (or media type) tells the client (browser, API consumer, etc.) how to process the data it receives
- type/subtype format
 - type/subtype; parameter=value

MIME types, subtypes and parameters

Top-Level Type	Description	Examples
text	Textual data, readable by humans.	text/plain, text/html
application	Data for applications to process.	application/json, application/pdf
image	Image files.	image/png, image/jpeg
audio	Audio files.	audio/mpeg, audio/ogg
video	Video files.	video/mp4, video/webm
multipart	Multiple parts in one message (e.g., emails).	multipart/form-data
message	Encapsulated messages.	message/http
font	Font files.	font/woff, font/woff2
model	3D models or CAD files.	model/3mf, model/gltf+json

- Subtypes examples
 - application/json → JSON data.
 - text/css → CSS stylesheet.
 - image/svg+xml → SVG image in XML format.
- Parameters example
 - text/html; charset=UTF-8

Common Request Headers

POST / HTTP/1.1

Host: localhost:8000

User-Agent: Mozilla/5.0 (Macintosh;...)... Firefox/51.0

Accept: text/html,application/xhtml+xml,..., */*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Connection: keep-alive

Upgrade-Insecure-Requests: 1

Content-Type: multipart/form-data; boundary=-12656974

Content-Length: 345

-12656974

(more data)

If neither HTML nor XHTML is available, the client will accept any other media type (any type/any subtype), but with a lower preference.

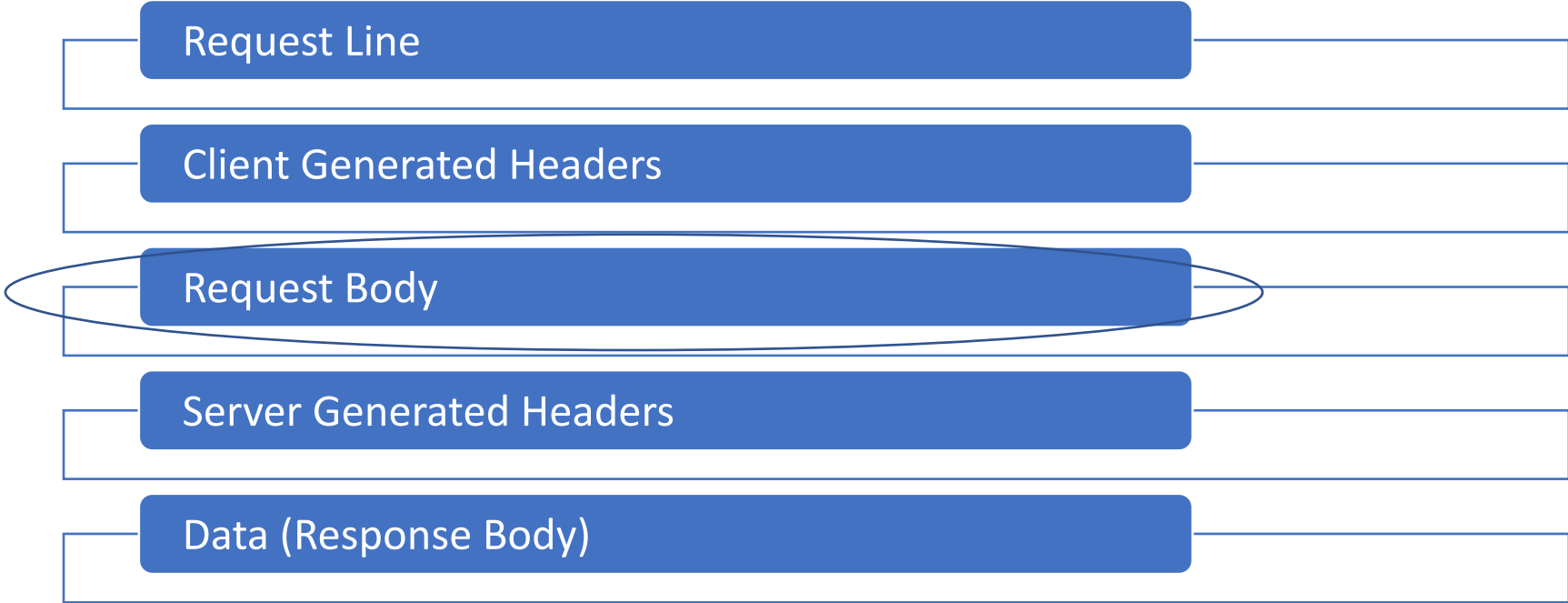
Request headers

General headers

Entity headers

A form is submitted with files, images, or binary data using the HTTP POST method
The boundary parameter is a unique delimiter that separates each part of the multipart message (auto-generated: unique enough not to occur in the actual form data)

Request Body



The diagram illustrates the structure of an HTTP request and response. It consists of five horizontal sections, each with a blue header bar and a white content box. The sections are: Request Line, Client Generated Headers, Request Body, Server Generated Headers, and Data (Response Body). The 'Request Body' section is highlighted with a blue oval. The 'Request Line' and 'Client Generated Headers' sections are connected by a vertical line on the left. The 'Server Generated Headers' and 'Data (Response Body)' sections are connected by a vertical line on the right.

Request Line

Client Generated Headers

Request Body

Server Generated Headers

Data (Response Body)

Request Body

- Payload
 - Mainly for sending data to server
 - Only PATCH, POST, and PUT requests have a body
 - Can be one or multiple files (single- Vs multiple-resource bodies)
-
- Examples
 - `name=FirstName+LastName&email=bsmth%40example.com`
 - JSON messages
 - Multiple parts separated by a delimiter

Server Generated Headers



The diagram illustrates the structure of an HTTP request. It consists of five horizontal blue bars, each representing a different part of the request. From top to bottom, these are: 'Request Line', 'Client Generated Headers', 'Request Body', 'Server Generated Headers', and 'Data (Response Body)'. Each bar is connected to a thin blue line that extends to the right, forming a continuous structure. The 'Server Generated Headers' bar is highlighted with a blue oval.

Request Line

Client Generated Headers

Request Body

Server Generated Headers

Data (Response Body)

Start (status) line

- Syntax: <protocol> <status-code> <reason-phrase>
 - Example: HTTP/1.1 201 Created

Status code ranges

Code Range	Meaning
100's	Informational
200's	Success
300's	Redirected
400's	Request Incomplete
500's	Server Error

Examples

Status code	Reason phrase
200	OK
301	Moved permanently
302	Moved temporarily
304	Not modified
403	Forbidden
404	Not found
500	Internal server error

Common Response Headers

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

Connection: Keep-Alive

Content-Encoding: gzip

Content-Type: text/html; charset=utf-8

Date: Wed, 10 Aug 2016 13:17:18 GMT

Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"

Keep-Alive: timeout=5, max=999

Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT

Server: Apache

Set-Cookie: csrftoken=.....

Transfer-Encoding: chunked

Vary: Cookie, Accept-Encoding

X-Frame-Options: DENY

Response headers

Entity headers

General headers

(body)

Common Response Headers

HTTP/1.1 200 OK

```
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 10 Aug 2016 13:17:18 GMT
Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"
Keep-Alive: timeout=5, max=999
Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT
Server: Apache
Set-Cookie: csrftoken=.....
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding
X-Frame-Options: DENY
```

Response headers

Entity headers

General headers

If no data is sent or received within the 5-second period, the connection will be closed.
The server allows up to 999 requests to be sent over the same connection before it will be closed, regardless of the timeout setting.

(body)

security feature that prevents a webpage from being embedded inside a frame

The content of this response may change depending on the specified headers in future requests.

Response Body



The diagram illustrates the structure of an HTTP request, consisting of five stacked components. Each component is represented by a blue rounded rectangle with its name written inside, followed by a white rectangular box to its right. A vertical line on the left side of the first four components connects them. The fifth component, 'Data (Response Body)', is highlighted with a blue oval.

Request Line

Client Generated Headers

Request Body

Server Generated Headers

Data (Response Body)

Response Body

- Payload
- Responses with a status code that sufficiently answers the request without the need for corresponding payload (like 201 Created or 204 No Content) usually don't carry a payload.
- Categories:
 - Single-resource bodies, consisting of a single file of known length, defined by the two headers: *Content-Type* and *Content-Length*.
 - Single-resource bodies, consisting of a single file of unknown length, encoded by chunks with *Transfer-Encoding* set to chunked.
 - Multiple-resource bodies, consisting of a multipart body, each containing a different section of information. These are relatively rare.

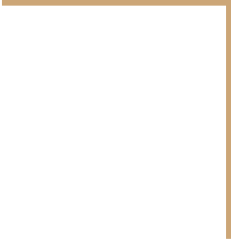


Other Important Concepts



Other Important Concepts

- Cookies
- Single- Vs Cross-origin policy
- Other categories of headers
- HTTP/2
- Chunked Transfer encoding



COOKIES

Cookies

- Small piece of data that a server sends to the user's web browser
- The browser may store it and send it back with later requests to the same server
- Applications
 - Record information about the client's usage patterns, including the date and time the client visited the site, which pages were accessed, Web browser preferences, etc
 - It remembers stateful information for the stateless HTTP protocol.
- Lifetime
 - Session cookies are deleted when the "current session" ends
 - The browser defines when the "current session" ends, and some browsers use session restoring when restarting, which can cause session cookies to last indefinitely long
 - Permanent cookies are deleted at a date specified by the Expires attribute, or after a period of time specified by the Max-Age attribute. E.g.
 - `Set-Cookie: id=a3fWa; Expires=Wed, 31 Oct 2021 07:28:00 GMT;`

Cookies Headers

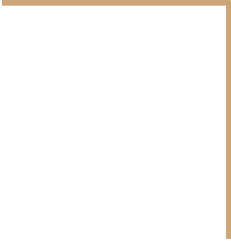
- Cookie

- Contains stored HTTP cookies previously sent by the server with the Set-Cookie header.

- Set-Cookie

- Send cookies from the server to the user-agent.





CORS

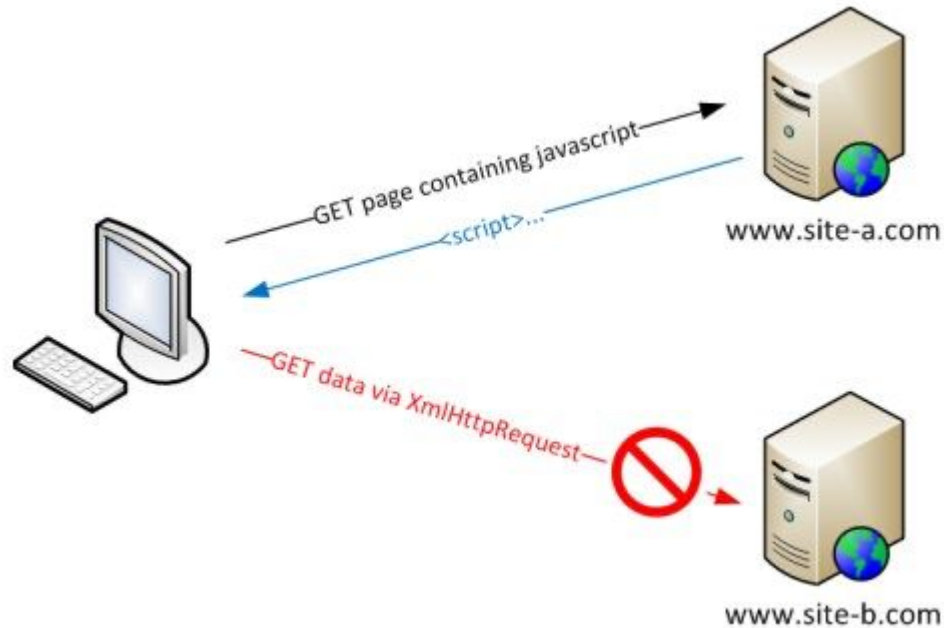


Same Origin Policy

- origin: "scheme/host/port tuple"
 - Two URLs have the same origin if the tuple is the same for both
- Critical security mechanism that restricts how a document or script loaded by one origin can interact with a resource from another origin.
- For example, it prevents a malicious website on the Internet from running JS in a browser to read data from a third-party webmail service (which the user is signed into).

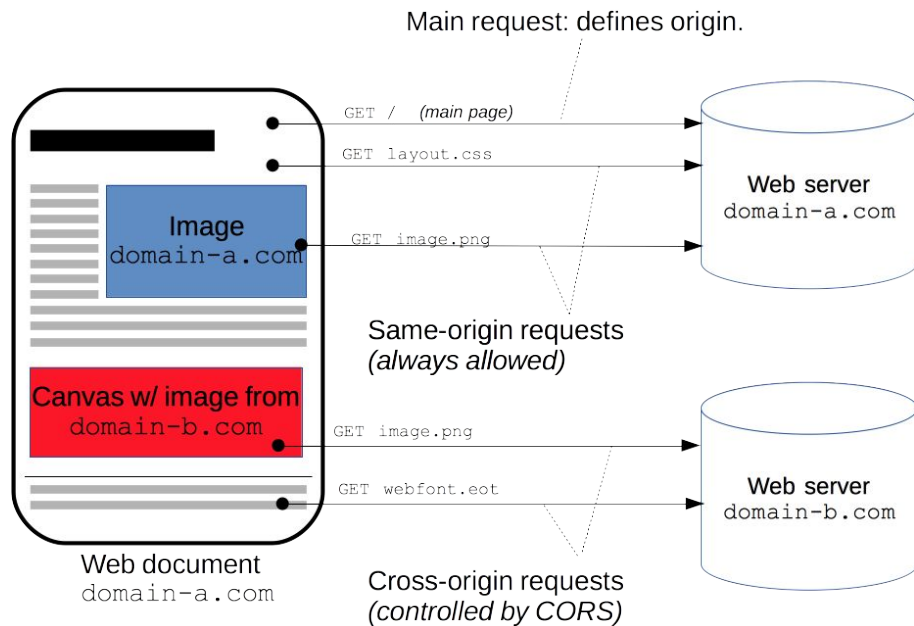
Cross-Origin HTTP Requests

- For security reasons, browsers restrict cross-origin HTTP requests initiated from scripts



Cross-Origin Resource Sharing (CORS)

- Protocol that is implemented with the help of specific HTTP headers
- Cross-origin HTTP requests are managed through CORS
- Two types:
 - Simple requests
 - Preflighted requests



Source: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

CORS 'simple requests'

- Server responds with the Access-Control-Allow-Origin header
- E.g.
 - Access-Control-Allow-Origin: <https://foo.example>
 - Access-Control-Allow-Origin: *



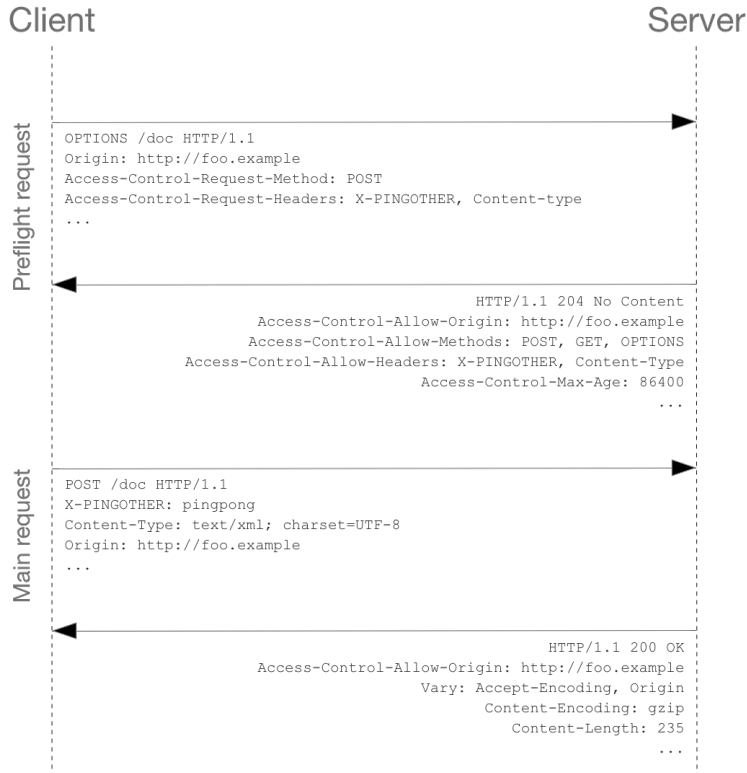
CORS 'simple requests' (Conditions)

- One of the allowed methods: GET, HEAD, POST
- Headers automatically set by the user agent (e.g. Connection, User-Agent)
- CORS-safelisted request-headers
 - Accept
 - Accept-Language
 - Content-Language
 - Content-Type (but note the additional requirements below)
 - DPR
 - Downlink
 - Save-Data
 - Viewport-Width
 - Width
- The only allowed values for the Content-Type header are:
 - application/x-www-form-urlencoded
 - multipart/form-data
 - text/plain
- No event listeners
- No ReadableStream object is used in the request

CORS 'preflighted requests'

- By default used in cross-site requests
- Intended requests are preceded by a *OPTIONS* HTTP request to the resource on the other domain, to determine if the actual request is safe to send
- Server responds to the *OPTIONS* HTTP Request with a set of headers defining the safe origins, methods, headers and the seconds that the preflighted request will be cached. E.g.
 - Access-Control-Allow-Origin: http://foo.example
 - Access-Control-Allow-Methods: POST, GET, OPTIONS
 - Access-Control-Allow-Headers: X-PINGOTHER, Content-Type
 - Access-Control-Max-Age: 86400

CORS 'preflighted requests'



How the 'Origin' value is known?

- Referrer header
 - Refers to the web resource that provided the endpoint to the resource to which the HTTP request is addressed
- In the previous example the POST request would look like that:

```
1 POST /doc HTTP/1.1
2 Host: bar.other
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:71.0) Gecko/20100101 Firefox/71.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-us,en;q=0.5
6 Accept-Encoding: gzip,deflate
7 Connection: keep-alive
8 X-PINGOTHER: pongpong
9 Content-Type: text/xml; charset=UTF-8
10 Referrer: https://foo.example/examples/preflightInvocation.html
11 Content-Length: 55
12 Origin: https://foo.example
13 Pragma: no-cache
14 Cache-Control: no-cache
15
16 <person><name>Arun</name></person>
17
```



Other categories of headers



Caching Headers

- *Age*
 - The time, in seconds, that the object has been in a proxy cache.
- *Cache-Control*
 - Directives for caching mechanisms in both requests and responses.
- *Clear-Site-Data*
 - Clears browsing data (e.g. cookies, storage, cache) associated with the requesting website.
- *Expires*
 - The date/time after which the response is considered stale.
- *Warning*
 - General warning information about possible problems.

Caching Headers Example

Client loads content received earlier

URL	Method	Result	Type	Received	Taken	Initiator
http://galaxy.hua.gr/~tserpes/index.html	GET	304	text/html	179 B	172 ms	refresh
/~tserpes/style.css	GET	304	text/css	176 B	62 ms	<link rel="style...
/~tserpes/ga.js	GET	304	application/javascript	190 B	31 ms	<script>
/~tserpes/jquery-1.8.js	GET	304	application/javascript	195 B	62 ms	<script>
/~tserpes/pageLoader.js	GET	304	application/javascript	190 B	78 ms	<script>
http://www.google-analytics.com/ga.js	GET	304	text/javascript	225 B	156 ms	<script>
/~tserpes/header.jpg	GET	304	image/jpeg	181 B	46 ms	background-image
http://www.google-analytics.com/__utm...	GET	200	image/gif	376 B	172 ms	

Conditional Headers

- *Last-Modified*
 - The last modification date of the resource, used to compare several versions of the same resource
- *ETag*
 - A unique string identifying the version of the resource
- *If-Match*
 - Makes the request conditional, and applies the method only if the stored resource matches one of the given ETags.
- *If-None-Match*
 - ...
- *If-Modified-Since*
 - Makes the request conditional, and expects the entity to be transmitted only if it has been modified after the given date. This is used to transmit data only when the cache is out of date.
- *If-Unmodified-Since*
 - ...
- *Vary*
 - Determines how to match request headers to decide whether a cached response can be used rather than requesting a fresh one from the origin server.

Chunked transfer encoding

Behavior

- A data transfer mechanism in which data is sent in blocks called "chunks"
- It uses the **Transfer-Encoding** header in place of the **Content-Length** header, the sender does not need to know the length of the content before it starts transmitting a response to the receiver. (useful for dynamically-generated content).
- Size is sent before the chunk so that the receiver can tell when it has finished receiving data for that chunk.
- Data transfer is terminated by a final chunk of length zero.

Benefits

- Allows a server to maintain an HTTP persistent connection for dynamically generated content
- Allows the sender to send header fields after the message body, in cases where values cannot be known until the content has been produced (e.g., digital signature)



HTTP/2

HTTP/2

- RFC 7540 in May 2015
- Supported by the 49% of the top 10 million websites supported HTTP/2 (Source: [Usage of HTTP/2 for websites](#), October 2020)
- Create a negotiation mechanism that allows clients and servers to elect to use HTTP 1.1, 2.0, or potentially other non-HTTP protocols
- Maintain high-level compatibility with HTTP 1.1 (for example with methods, status codes, URIs, and most header fields)
- Decrease latency to improve page load speed in web browsers by considering:
 - Data compression of HTTP headers
 - HTTP/2 Server Push
 - Pipelining of requests
 - Fixing the head-of-line blocking problem in HTTP 1.x
 - Multiplexing multiple requests over a single TCP connection
- Adaptation to existing use cases of HTTP, such as desktop web browsers, mobile web browsers, etc.

Data Compression of HTTP Headers

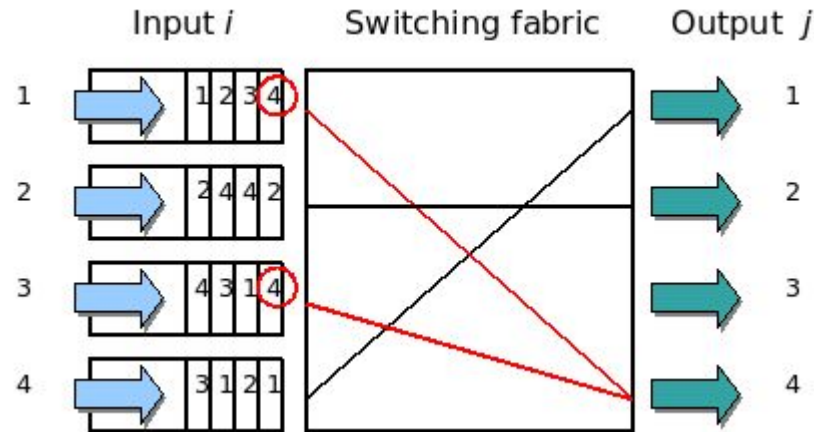
- HTTP/1.x messages performance drawbacks
 - Headers, unlike bodies, are uncompressed.
 - Headers are often very similar from one message to the next one, yet still repeated across connections.
 - No multiplexing can be done. Several connections need opening on the same server: and warm TCP connections are more efficient than cold ones.
- HTTP/2 introduces an extra step: it divides HTTP/1.x messages into frames which are embedded in a stream. Data and header frames are separated, this allows header compression. Several streams can be combined together, a process called multiplexing, allowing more efficient underlying TCP connections.

HTTP/2 Server Push

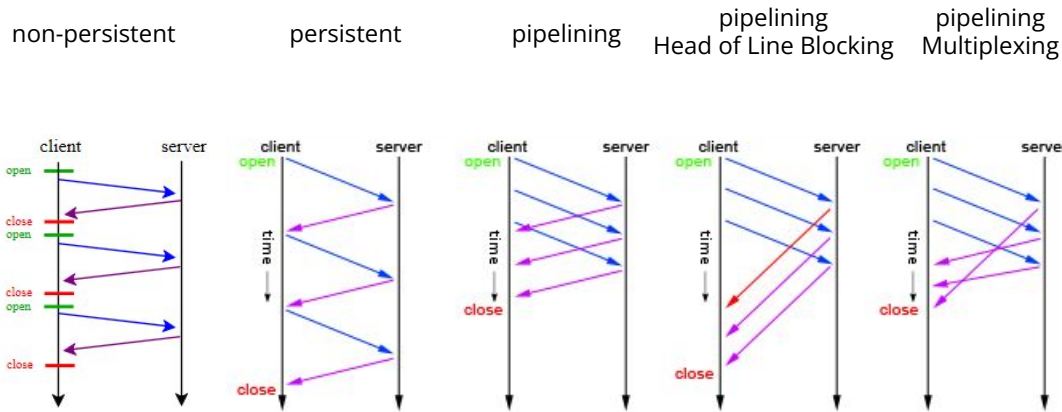
- Allows a web server to send resources to a web browser before the browser gets to request them. E.g.,
 - A website with two resources. Once resourceA is received it contains instructions to also request resourceB.
 - With HTTP/2 Push, the server can send resourceB immediately after resourceA
 - The client will retrieve it locally when it needs it

HTTP Head-of-Line Blocking

- In the example below
 - Input₁ or Input₃ have to wait one another
 - If Input₁ yields, then it will block Input₂ on round #2



HTTP Pipelining



"open"- "close" indicate the duration during which the TCP connection remains open

Testing

Testing

- Curl (command line)
 - curl -I "<https://www.keycdn.com/support/popular-curl-examples>"
 - curl -I -H "if-modified-since:Mon, 26 Oct 2020 10:20:05 GMT" "<https://www.keycdn.com/support/popular-curl-examples>"
- Tools like POSTMAN
- Browser debugger

Resources

- <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
- <https://tools.ietf.org/html/rfc7540>
- <https://tools.ietf.org/html/rfc3986>